
Write-Up: Crack The Hash

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe

Dificuldade: Fácil

Data de resolução: 10/09/2026

Autor: Luís Eduardo Curi Serra

Matrícula: 251033574

Professor: Roberto Rodrigues Filho

Sumário

1	Visão Geral	2
2	Reconhecimento: Identificação e Ferramentas	2
2.1	Ferramenta de quebra — hashcat	2
3	Exploração: Quebra dos Hashes	3
3.1	Nível 1 — (Task 1)	3
3.1.1	#1 — MD5	4
3.1.2	#2 — SHA-1	4
3.1.3	#3 — SHA-256	4
3.1.4	#4 — bcrypt	4
3.1.5	#5 — MD4	5
3.2	Nível 2 — (Task 2)	5
3.2.1	#1 — SHA-256 (com regras)	6
3.2.2	#2 — NTLM	6
3.2.3	#3 — sha512crypt (\$6\$)	6
3.2.4	#4 — HMAC-SHA1 (chave = sal)	7
3.3	Hashes Adicionais (exercício complementar)	7
3.3.1	Add #1 — MD5	8
3.3.2	Add #2 — SHA-1	8
3.3.3	Add #3 — MD4 (máscara)	8
3.3.4	Add #4 — HMAC-SHA1	9
3.3.5	Add #5 — SHA-256 (máscara)	9
4	Conclusão	9
5	Referências	9

1 Visão Geral

Crack The Hash é uma sala de *criptografia* do TryHackMe, de dificuldade *fácil*. Diferentemente de uma máquina, não há serviço a explorar nem escalada de privilégio: o desafio entrega uma série de *hashes* e o objetivo é **identificar o algoritmo** de cada um e **recuperar o texto-claro**. A sala está dividida em dois níveis de dificuldade crescente, além disso foram resolvidos **hashes adicionais** propostos como exercício complementar. A Tabela ??

O fluxo aplicado a cada hash é sempre o mesmo: (i) **identificar** o algoritmo pelo comprimento e formato do hash, com apoio de *hashid*, *hash-identifier* e do *Magic* do CyberChef; (ii) **mapear o modo** correspondente do *hashcat* (-m); e (iii) **atacar** com o ataque adequado (-a) — dicionário (*rockyou.txt*), dicionário com regras de mutação, ou máscara/força-bruta quando há dica sobre o formato da senha. Como os hashes são a entrada pública do desafio e as senhas recuperadas são as próprias respostas didáticas da sala, ambos são exibidos em claro neste documento.

2 Reconhecimento: Identificação e Ferramentas

O “reconhecimento” de um desafio de *cracking* consiste em determinar, para cada hash, qual algoritmo o gerou. O comprimento em caracteres hexadecimais é a primeira pista (32 → família MD5/MD4; 40 → SHA-1; 64 → SHA-256); prefixos como \$2y\$ (bcrypt) e \$6\$ (sha512crypt) são identificadores diretos do formato.

- **hashid** — lista os candidatos de algoritmo e já informa o modo (-m) do *hashcat*: `hashid -m '<hash>'`.
- **hash-identifier** — ferramenta interativa equivalente.
- **CyberChef (*Magic*)** — detecta o tipo e sugere a família de algoritmos a partir do formato.

Um ponto essencial é que ferramentas de identificação retornam *várias* possibilidades para um mesmo comprimento (p. ex., um hash de 32 hex pode ser MD5, MD4, NTLM, RIPEMD-128 etc.). O procedimento correto é testar os candidatos em ordem de probabilidade até um deles quebrar — foi exatamente o que distinguiu os casos mais trabalhosos (Nível 1 #5 e Nível 2 #2).

2.1 Ferramenta de quebra — hashcat

A quebra propriamente dita foi feita com *hashcat*. Os dois parâmetros centrais são o **modo** (-m, o algoritmo) e o **tipo de ataque** (-a). A Tabela 1 reúne os modos efetivamente usados no desafio.

Tabela 1: Modos de `hashcat` utilizados neste desafio.

-m	Algoritmo	Onde apareceu
0	MD5	N1#1, Add#1
100	SHA-1	N1#2, Add#2
900	MD4	N1#5, Add#3
1000	NTLM	N2#2
1400	SHA-256	N1#3, N2#1, Add#5
1800	sha512crypt (\$6\$)	N2#3
3200	bcrypt (\$2*\$)	N1#4
160	HMAC-SHA1 (chave = sal)	N2#4, Add#4

Tipos de ataque (-a) empregados: **0** (dicionário), **3** (máscara / força-bruta). Recursos auxiliares:

- **Dicionário:** `/usr/share/wordlists/rockyou.txt`.
- **Regras** (-r): geram variações de cada palavra do dicionário (maiúsculas, números no fim, *leets-peak*), usadas quando a senha é uma palavra comum mutada. Empregou-se `best66.rule`, nativo do *Kali Linux*.
- **Máscaras** (-a 3): `?d`=dígitos, `?l`=minúsculas, `?u`=maiúsculas. Com `-1` define-se um *charset* próprio referenciado por `?1` — útil quando cada posição pode ser de vários conjuntos.
- **Sal:** para hashes com sal, o `hashcat` recebe o par no formato `hash:sal` (modos 110/120/160), ou o sal já vem embutido no próprio hash (formato `$6$<sal>$<hash>`).
- **Otimização** (-0): habilita o *kernel* otimizado (limita o tamanho da senha, mas acelera bastante).

3 Exploração: Quebra dos Hashes

Para cada hash apresenta-se: o valor de entrada, o algoritmo identificado e o respectivo modo, o comando de quebra e o texto-claro recuperado. Onde houve tentativas frustradas relevantes, elas são registradas em destaque, pois compõem o raciocínio de identificação.

3.1 Nível 1 — (Task 1)

Tabela 2: Resumo do Nível 1.

Hash (abrev.)	Algoritmo	-m	Ataque	Texto-claro
48bb...ed4d	MD5	0	dicionário	easy
CBFD...2A97	SHA-1	100	dicionário	password123
1C8B...3032	SHA-256	1400	dicionário	letmein
\$2y\$12\$...	bcrypt	3200	dicionário (4 letras)	bleh
2794...3daa	MD4	900	dicionário + regras	Eternity22

3.1.1 #1 — MD5

Hash de 32 hex: 48bb6e862e54f2a795ffc4e541caed4d. O CyberChef aponta a família MD (MD5, MD4, RIPEMD-128...); começa-se pelo mais provável, MD5 (-m 0), com dicionário.

```
hashcat -m 0 -a 0 t1_1.txt /usr/share/wordlists/rockyou.txt
```

Listing 1: Quebra do hash MD5 por dicionário.

Texto-claro: **easy**.

3.1.2 #2 — SHA-1

Hash de 40 hex: CBFDAC6008F9CAB4083784CBD1874F76618D2A97. O comprimento identifica SHA-1 (-m 100).

```
hashcat -m 100 -a 0 t1_2.txt /usr/share/wordlists/rockyou.txt
```

Listing 2: Quebra do hash SHA-1 por dicionário.

Texto-claro: **password123**.

3.1.3 #3 — SHA-256

Hash de 64 hex: 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032. Comprimento típico de SHA-256 (-m 1400).

```
hashcat -m 1400 -a 0 t1_3.txt /usr/share/wordlists/rockyou.txt
```

Listing 3: Quebra do hash SHA-256 por dicionário.

Texto-claro: **letmein**.

3.1.4 #4 — bcrypt

Hash \$2y\$12\$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom. O prefixo \$2y\$ e o fator de custo 12 identificam bcrypt (-m 3200) via hashid. Por ser um algoritmo deliberadamente lento (2¹² iterações por candidato), o dicionário completo é inviável em tempo hábil; por isso o rockyou foi filtrado para conter apenas palavras alfabéticas de **4 caracteres**, reduzindo drasticamente o espaço de busca.

```
grep -xE '[A-Za-z]{4}' /usr/share/wordlists/rockyou.txt > words4.txt  
hashcat -m 3200 -a 0 t1_4.txt words4.txt
```

Listing 4: Filtragem do dicionário e quebra do bcrypt.

Texto-claro: **bleh**.

3.1.5 #5 — MD4

Hash de 32 hex: 279412f945939ba78ce0758d3fd83daa. Foi o hash mais trabalhoso do Nível 1: o texto-claro (**Eternity22**) não está no rockyou puro e o algoritmo não é MD5. Após descartar MD5 e SHA-1, confirma-se **MD4** (-m 900); a senha é uma palavra do dicionário *mutada*, recuperada com o dicionário mais o conjunto de regras best66.rule.

```
hashcat -m 900 -a 0 t1_5.txt /usr/share/wordlists/rockyou.txt \
-r /usr/share/hashcat/rules/best66.rule
```

Listing 5: Quebra do hash MD4 com dicionário + regras.

Texto-claro: **Eternity22**.

Tentativas sem sucesso (becos sem saída)

- MD5 (-m 0) e MD4 (-m 900) com rockyou *puro*: sem resultado — a senha é uma mutação, não uma palavra literal.
- Máscara de força-bruta com 10 minúsculas (-a 3 ?l?l?l?l?l?l?l?l?l?l): estimativa de ≈ 5 dias mesmo com -0 — inviável, abandonado (e a senha nem é toda minúscula).
- Lista externa *fortinet-2021* (SecLists): também sem resultado.
- Só a combinação **MD4 + rockyou + regras** destravou.

3.2 Nível 2 — (Task 2)

Tabela 3: Resumo do Nível 2.

Hash (abrev.)	Algoritmo	-m	Ataque	Texto-claro
F09E...0C85	SHA-256	1400	dicionário + regras	paule
1DFE...CC1B	NTLM	1000	dicionário + regras	n63umy8lkf4i
\$6\$...As02.	sha512crypt	1800	dicionário (6 chars)	waka99
e5d8...56d6 : sal	HMAC-SHA1	160	dicionário + regras	481616481616

3.2.1 #1 — SHA-256 (com regras)

Hash de 64 hex: F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85 (SHA-256, -m 1400). O rockyou puro não encontra; a senha é uma mutação, quebrada com regras.

```
hashcat -m 1400 -a 0 t2_1.txt /usr/share/wordlists/rockyou.txt \  
-r /usr/share/hashcat/rules/best66.rule
```

Listing 6: SHA-256 com dicionário + regras.

Texto-claro: **paule**.

3.2.2 #2 — NTLM

Hash de 32 hex: 1DFECA0C002AE40B8619ECF94819CC1B. O hashid retorna uma lista extensa (MD5, MD4, *Double MD5*, LM, NTLM...). Testando os candidatos em ordem, todos falham até **NTLM** (-m 1000), que quebra. NTLM é o hash de senhas do Windows.

```
hashcat -m 1000 -a 0 t2_2.txt /usr/share/wordlists/rockyou.txt \  
-r /usr/share/hashcat/rules/best66.rule
```

Listing 7: NTLM com dicionário + regras.

Texto-claro: **n63umy8lkf4i**.

Tentativas sem sucesso (becos sem saída)

Candidatos testados antes do NTLM, todos sem sucesso:

- MD5 (-m 0) e MD4 (-m 900) — com -0 e regras.
- *Double MD5* (-m 2600) e LM (-m 3000).
- *Lotus Notes/Domino 5* (-m 8600): estimativa de ≈ 30 min — descartado por custo/benefício.

3.2.3 #3 — sha512crypt (\$6\$)

Hash no formato \$6\$<sal>\$<hash>:

```
$6$aReallyHardSalt$6WKUTqzq.UQQmrm0p/T7MPpMbGNnzXPMAXi4bJmL9be.cfi3/qxIf.  
→ hsGpS41BqMhSrHVXgMpdjS6xeKZAs02.
```

O prefixo \$6\$ identifica sha512crypt (-m 1800), o esquema do /etc/shadow moderno; o sal (aReallyHardSalt) já está embutido no próprio hash, sem necessidade de informá-lo à parte. Por

ser um algoritmo com milhares de iterações (muito lento), o **rockyou** foi filtrado para palavras alfanuméricas de **6 caracteres**; mesmo assim a quebra levou ≈ 10 minutos.

```
grep -xE '[A-Za-z0-9]{6}' /usr/share/wordlists/rockyou.txt > words6.txt  
hashcat -m 1800 -a 0 t2_3.txt words6.txt -O
```

Listing 8: Filtragem por 6 caracteres e quebra do sha512crypt.

Texto-claro: **waka99**.

3.2.4 #4 — HMAC-SHA1 (chave = sal)

Par hash:sal: e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme. O hashid sugere a família SHA-1. A questão é *como* o sal entra: após descartar as concatenações simples (senha+sal e sal+senha), o hash corresponde a **HMAC-SHA1 com o sal como chave** (-m 160).

```
hashcat -m 160 -a 0 t2_4.txt /usr/share/wordlists/rockyou.txt \  
-r /usr/share/hashcat/rules/best66.rule
```

Listing 9: HMAC-SHA1 (modo 160) com o sal como chave.

Texto-claro: **481616481616**.

Tentativas sem sucesso (becos sem saída)

- -m 110 — sha1(senha.sal): sem resultado.
- -m 120 — sha1(sal.senha): sem resultado.
- -m 160 — HMAC-SHA1(chave = sal): correspondência.

Lição: com sal, não basta o algoritmo-base — é preciso descobrir o esquema de combinação (concatenação vs. HMAC, e a ordem/chave).

3.3 Hashes Adicionais (exercício complementar)

Conjunto extra proposto como prática, aplicando o mesmo fluxo. Alguns vêm com **dica de formato**, o que favorece o ataque por *máscara*.

Tabela 4: Resumo dos hashes adicionais.

Hash (abrev.)	Algoritmo	-m	Ataque	Texto-claro
482c...1e38	MD5	0	dicionário	password123
861c...5228	SHA-1	100	dicionário	easy
4149...4099	MD4	900	máscara	0ffs3c
cdeb...5875 : sal	HMAC-SHA1	160	dicionário	ovelha
362f...2912	SHA-256	1400	máscara	sawctf

3.3.1 Add #1 — MD5

482c811da5d5b4bc6d497ffa98491e38 (MD5, -m 0).

```
hashcat -m 0 -a 0 add1.txt /usr/share/wordlists/rockyou.txt
```

Texto-claro: **password123**.

3.3.2 Add #2 — SHA-1

861c4f67e887dec85292d36ab05cd7a1a7275228 (SHA-1, -m 100).

```
hashcat -m 100 -a 0 add2.txt /usr/share/wordlists/rockyou.txt
```

Texto-claro: **easy**.

3.3.3 Add #3 — MD4 (máscara)

4149c5cc4c378444d116d65ad5ba4099, com dica “*apenas letras e números, 6 caracteres*”. Isso pede um ataque de **máscara** (-a 3) com um *charset* customizado que una dígitos, maiúsculas e minúsculas (-1 ?d?u?l). A máscara MD5 esgota sem achar; o algoritmo correto é **MD4** (-m 900).

```
hashcat -m 900 -a 3 add3.txt -1 ?d?u?l ?1?1?1?1?1?1
```

Listing 10: Máscara de 6 posições (?d/?u/?l) sobre MD4.

Texto-claro: **0ffs3c**.

Tentativas sem sucesso (becos sem saída)

A mesma máscara sobre MD5 (-m 0) esgotou o espaço sem correspondência.

3.3.4 Add #4 — HMAC-SHA1

Par `cdeb746ec095149627348b61d4140fc58b745875:satech`, com dica explícita **HMAC-SHA1** (`-m` $\rightarrow$ 160, sal como chave).

```
hashcat -m 160 -a 0 add4.txt /usr/share/wordlists/rockyou.txt
```

Texto-claro: **ovelha**.

3.3.5 Add #5 — SHA-256 (máscara)

`362fda2183b7ac73400a83f6ab2c359451e48adf6c3d46a2963ee2abdf852912` (64 hex, SHA-256, `-m` $\rightarrow$ 1400), com dica “*minúsculas e números, 6 caracteres*”. O *charset* customizado restringe-se a `?d?l`, o que torna a máscara pequena e rápida.

```
hashcat -m 1400 -a 3 add5.txt -1 ?d?l ?1?1?1?1?1?1
```

Listing 11: Máscara restrita a minúsculas e dígitos sobre SHA-256.

Texto-claro: **sawctf**.

4 Conclusão

O desafio reforça que a etapa decisiva do *cracking* é a **identificação** correta do algoritmo. O segundo fator é a **escolha da estratégia de ataque** segundo o custo do algoritmo e o que se sabe da senha: dicionário direto para palavras comuns; dicionário com **regras** para mutações (*Eternity22*, *paule*); **máscara** quando há dica de formato (*Offs3c*, *sawctf*); e, para algoritmos lentos (*bcrypt*, *sha512crypt*), **redução prévia do dicionário** por comprimento para tornar a quebra viável. Por fim, hashes com **sal** exigem descobrir o esquema de combinação (concatenação vs. HMAC), como no caso HMAC-SHA1 (modo 160). Por fim, todos os 14 hashes, 5 do Nível 1, 4 do Nível 2 e 5 adicionais, foram recuperados.

5 Referências

- TryHackMe — Sala *Crack The Hash* — <https://tryhackme.com/room/crackthehash>
- hashcat — *example hashes* (identificação de modos) — https://hashcat.net/wiki/doku.php?id=example_hashes
- hashcat — documentação (Kali Tools) — <https://www.kali.org/tools/hashcat/>

- CyberChef (função *Magic*) — <https://gchq.github.io/CyberChef/>
- SecLists — *Password lists* — <https://github.com/danielmiessler/SecLists>